AMERICAN UNIVERSITY OF PHNOM PENH

STUDY LOCALLY. LIVE GLOBALLY.

COSC 251 Fall 2026: Assignment 4 - Recursion (50 points)
Deadline: Nov 11 11:59 pm
Prepared by: Visethboti Sin (v.sin@aupp.edu.kh)

Overview: For this assignment, you are to implement the *RecursiveSolution* class, and the *IterativeSolution* class which contain methods to solve a list of provided problems. For each problems, you are to implement two methods for solving it, one method should be a recursive solution and another method should be an iterative solution. These methods should be under the *RecursiveSolution* class if they are recursive solutions, and the *IterativeSolution* class if they are iterative solutions.

Consider the time complexity when implementing each method to get full points for this assignment.

Project Requirements:

1. Solve the following problems:
   (a) Implement the method $tri(int\ n)$ that take an input n and return the sum of all non-negative integers up to n.
   (b) Implement the method $fact(int\ n)$ that take an input n and return the multiplication of all non-negative intergers up to n.
   (c) Implement the method $fib(int\ n)$ that takes an input n and return the nth Fibonacci number. The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, starting with 0 and 1.
   (d) Implement the method $reverseStr(String\ str)$ that takes a string as an input and returns its reverse.
   (e) Implement the method $hasChar(String\ str,\ char\ c)$ that takes a string and a char as inputs and returns true if the char exists in the string or false if it's not in the string.
   (f) Implement the method $gcd(int\ x1,\ int\ x2)$ to find the greatest common divisor of two non-negative integers n and m.
   (g) Implement the method $countUniquePaths(int\ n,\ int\ m)$ that takes two inputs n and m that are non-negative integers and returns the number of unique paths from the top left corner to the bottom right corner of a $n * m$ grid. Constraints: the path can only move down or right by one unit at a time.
2. Implement a recursive solution and an iterative solution for each of the problems listed above as methods in the *RecursiveSolution* class, and the *IterativeSolution* class.
3. Both classes may contain any number of other methods, as long as they are private.

What to turn-in:

This is a group assignment, each group should only have submission.

1. Your *RecursiveSolution* class with the required methods and functionalities. The file must be named RecursiveSolution.java and the class must be named RecursiveSolution.
2. Your *IterativeSolution* class with the required methods and functionalities. The file must be named IterativeSolution.java and the class must be named IterativeSolution.
3. Optional: any other supporting Java classes.

4. Note: Do not submit your Main Java class; your Java classes that you submit must not have any main method.

Grading:

1. Program Design (10): How well your solution is thought out.
2. Program Execution (15): The program runs correctly.
3. Specification Satisfaction (15): The program satisfies specifications completely and correctly.
4. Coding style and readability (10): Well-formatted, understandable code, and meaningful comments.